

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1711

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour

Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I MS Naveen (192371002) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Naveen.ms

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that: Constraints:
 - i. D1 cannot work Night shift
 - ii. D2 must work before D3 (Morning < Afternoon < Night)
 - iii. Only one doctor per shift
 - iv. D3 cannot work Morning
 - v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right
- ii. Cost of each move = 1
- iii. Cannot pass through obstacles
- iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information. The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- a) Analyze all the given constraints and explain how they affect the robot's decision-making process.
- b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- c) Demonstrate how the robot applies AI concepts such as:
 - Intelligent agents
 - Rationality
 - Environment type
- d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for	Provides justification with some clarity	Weak or unclear justification	No justification provided

		decisions with logical reasoning			
--	--	----------------------------------	--	--	--

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that: Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

Backtracking Search Solution

#Step 1: Define Variables and Domains

*Doctors: D1, D2, D3

*Shifts:

* Morning (M)

* Afternoon (A)

* Night (N)

*Constraints:

1. D1 cannot work Night.
2. D2 must work before D3 (Morning < Afternoon < Night).
3. Only one doctor per shift.
4. D3 cannot work Morning.
5. Every doctor is assigned exactly one shift.

Step 2: Backtracking Process

#Initial State

* D1 = ?

* D2 = ?

* D3 = ?

#Assign D1

Try *Morning ✓

* D1 = Morning

Remaining shifts: Afternoon, Night

#Assign D2

Try Afternoon* ✓

* D2 = Afternoon

Remaining shift: Night

#Assign D3

Only Night is available.

Check constraints:

- * D3 $\neq$ Morning ✓
- * D2 works before D3 (Afternoon < Night) ✓
- * One doctor per shift ✓

All constraints are satisfied.

Constraint Checks

Constraint	Status
-----	-----
D1 cannot work Night	✓ D1 = Morning
D2 before D3	✓ Afternoon < Night
One doctor per shift	✓
D3 cannot work Morning	✓ D3 = Night
Each doctor assigned one shift	✓

#Backtracking Tree

```
```text
Start
|
├── D1 = Morning ✓
| |
| ├── D2 = Afternoon ✓
| | |
| | └── D3 = Night ✓ (Valid Solution)
| |
| └──
|
└──
```
```

Since the first assignment satisfies all constraints, **no backtracking is required**.

Final Valid Schedule

| | | |
|-----------|--------|--|
| Shift | Doctor | |
| ----- | ----- | |
| Morning | D1 | |
| Afternoon | D2 | |
| Night | D3 | |

#Final Answer

- * Morning → D1
- * Afternoon → D2
- * Night → D3

This assignment satisfies all the given constraints.

4. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right
- ii. Cost of each move = 1
- v. Cannot pass through obstacles
- vi. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

#BFS Search Algorithm Solution

Assume the following **5×5 grid**.

```
```text
S . . X .
. X . X .
. X . . .
. . X X .
. . . . G
```
```

Where:

*S = Start (0,0)

*G= Goal (4,4)

*X = Obstacle

*.= Free cell

#Step 1: Initial State

*Start: (0,0)

*Goal:(4,4)

*Queue:[(0,0)]

*Visited: {(0,0)}

#Step 2: BFS Node Expansion

| Step | Expanded Node | Queue after Expansion |
|------|---------------|-----------------------|
|------|---------------|-----------------------|

| ---- | ----- | ----- |
|------|-------|-------|
|------|-------|-------|

| | | |
|---|-------|--------------|
| 1 | (0,0) | (1,0), (0,1) |
|---|-------|--------------|

| | | |
|---|-------|--------------|
| 2 | (1,0) | (0,1), (2,0) |
|---|-------|--------------|

| | | |
|---|-------|--------------|
| 3 | (0,1) | (2,0), (0,2) |
|---|-------|--------------|

| | | |
|---|-------|--------------|
| 4 | (2,0) | (0,2), (3,0) |
|---|-------|--------------|

| | | |
|---|-------|--------------|
| 5 | (0,2) | (3,0), (1,2) |
|---|-------|--------------|

| | | |
|---|-------|---------------------|
| 6 | (3,0) | (1,2), (4,0), (3,1) |
|---|-------|---------------------|

| | | |
|---|-------|---------------------|
| 7 | (1,2) | (4,0), (3,1), (2,2) |
|---|-------|---------------------|

| | | |
|---|-------|---------------------|
| 8 | (4,0) | (3,1), (2,2), (4,1) |
|---|-------|---------------------|

| | | |
|---|-------|--------------|
| 9 | (3,1) | (2,2), (4,1) |
|---|-------|--------------|

| | | |
|----|-------|--------------|
| 10 | (2,2) | (4,1), (2,3) |
|----|-------|--------------|

| | | |
|----|-------|--------------|
| 11 | (4,1) | (2,3), (4,2) |
|----|-------|--------------|

| | | |
|----|-------|--------------|
| 12 | (2,3) | (4,2), (2,4) |
|----|-------|--------------|

| | | | |
|----|-------|---------------------|--|
| 13 | (4,2) | (2,4), (4,3) | |
| 14 | (2,4) | (4,3), (3,4), (1,4) | |
| 15 | (4,3) | (3,4), (1,4), (4,4) | |
| 16 | (4,4) | Goal Reached | |

Priority Queue (BFS Queue) Updates

```text

Initial:

[(0,0)]

After expanding (0,0):

[(1,0), (0,1)]

After expanding (1,0):

[(0,1), (2,0)]

After expanding (0,1):

[(2,0), (0,2)]

...

Final:

[(4,4)] → Goal Found

...

---

#Manhattan Distance Heuristic

Formula:

$$*h(n) = |x_1 - x_2| + |y_1 - y_2|$$

Example:

For Start (0,0) and Goal (4,4):

$$h = |4-0| + |4-0| = 8$$

> **Note:** BFS does **not** use the Manhattan heuristic for node selection. The Manhattan distance heuristic is used by informed search algorithms such as **A**.

---

#Optimal Path

```
```text
```

(0,0)

→ (1,0)

→ (2,0)

→ (3,0)

→ (4,0)

→ (4,1)

→ (4,2)

→ (4,3)

→ (4,4)

```
```
```

```

```

```
#Path Cost
```

```
* Number of moves = **8**
```

```
* Cost per move = **1**
```

```
Total Cost = 8
```

```

```

```
#Final Answer
```

\*Algorithm: Breadth-First Search (BFS)

\*Optimal Path:  $S \rightarrow (1,0) \rightarrow (2,0) \rightarrow (3,0) \rightarrow (4,0) \rightarrow (4,1) \rightarrow (4,2) \rightarrow (4,3) \rightarrow G$

\*Total Cost: 8

\*Heuristic: Manhattan Distance =  $(|x_1 - x_2| + |y_1 - y_2|)$  (used in A\*; not used by BFS).